

Name	Value	Inherited	Watchable	Signature
NumPartsSinceSetup	0			-> integer
NumPred	1			-> integer
NumSucc	3			-> integer

Name of the read-only attribute Current value of the read-only attribute Data type of the return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

To query the value of a *read-only attribute*, you might, for example, type:

```
print MyDialog.UUID
```

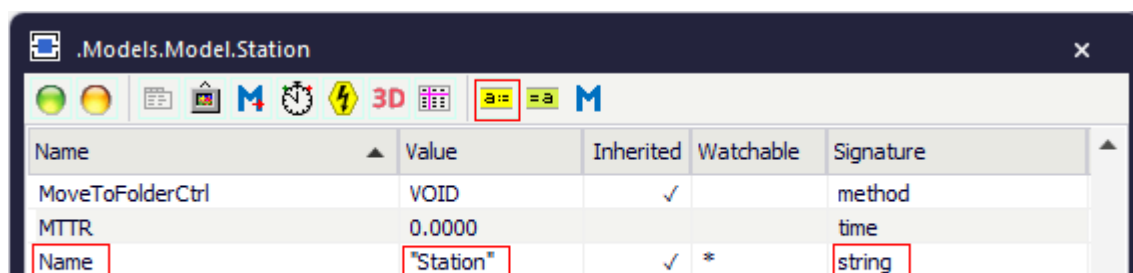
Attributes of the Dialog

_Attributes of the Dialog

The *Dialog* provides:

- The *attributes* listed in the table of contents to the left.
- The [Attributes of All Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.



Name	Value	Inherited	Watchable	Signature
MoveToFolderCtrl	VOID	✓		method
MTTR	0.0000			time
Name	"Station"	✓	*	string

Name of the attribute Current value of the attribute Data type of the assignment value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyDialog.Locked := true  
MyDialog.openDialog // show the effect
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyDialog.ArgumentForApply  
posit := Station.Cont.XPos
```

ArgumentForApply [SimTalk]

Designates an **Argument for Apply** for the **Callback Method** when the user clicks **Apply** or **OK** in the dialog box of the *Dialog* designated by <Path>.

Remarks

If the user clicks **OK**, the *Dialog* executes the **Callback Method** twice. The first time the **Apply** section is called. The second time, the **Close** section is called.

If the user clicks **Apply**, the *Dialog* only executes the **Apply** section of the **Callback Method**.

Type

Attribute

Syntax

```
<Path>.ArgumentForApply:string
```

Assignment Value

You can assign a value of data type *string*.

Examples

```
MyDialog.ArgumentForApply := "Apply"
```

```
param action : string
print "Current action: ",action
switch action
case "Open"
    @.setCaption("Number_Inp",to_str(Variable1))
case "Apply"
    var str:string := @.getValue("Number_Inp")
    Variable1 := str_to_num(str)
case "Close"
end
```

SimTalk

[CallbackMethod \[SimTalk\] - Dialog](#)

See also

[Argument for Apply \[text box\]](#)

[Callback Method \[text box\] - Dialog](#)

ArgumentForClose [SimTalk]

Designates an **Argument for Close** for the **Callback Method** when the user clicks **Cancel** in the dialog of the *Dialog* designated by <Path> or the close button in its title bar.

Remarks

If the user clicks **OK**, the *Dialog* executes the **Callback Method** twice. The first time the **Apply** section is called. The second time, the **Close** section is called.

If the user clicks **Apply**, the *Dialog* only executes the **Apply** section of the **Callback Method**.

Type

Attribute

Syntax

```
<Path>.ArgumentForClose:string
```

Assignment Value

You can assign a value of data type *string*.

Example

```
MyDialog.ArgumentForClose := "Close"
```

SimTalk

[CallbackMethod \[SimTalk\] - Dialog](#)

See also

[Argument for Close \[text box\]](#)

[Callback Method \[text box\] - Dialog](#)

ArgumentForOpen [SimTalk]

Designates an **Argument for Open** for the **Callback Method** when the user opens the *Dialog* designated by <Path>.

Remarks

You might, for example, use it to preallocate data in the dialog box.

Type

Attribute

Syntax

```
<Path>.ArgumentForOpen:string
```

Assignment Value

You can assign a value of data type *string*.

Example

```
MyDialog.ArgumentForOpen := "Open"
```

SimTalk

[CallbackMethod \[SimTalk\] - Dialog](#)

See also

[Argument for Open \[text box\]](#)

[Callback Method \[text box\] - Dialog](#)

CallbackMethod [SimTalk] - Dialog

Sets the path to a *Method* that the dialog items in the *Dialog* designated by <Path> triggers.

Type

Attribute

Syntax

```
<Path>.CallbackMethod:object
```

Parameters

The **Callback Method** can contain the following parameters:

- The **Open** section initializes the contents of the dialog box or sets the dialog items to values of your choice.
- The *Dialog* executes the **Apply** section when the user clicks **OK** or **Apply** in the dialog box of the *Dialog* you define. The source code you enter may evaluate new or changed values.
- The *Dialog* executes the **Close** section when the user clicks **Cancel** in the dialog box of the *Dialog* you define, or when he closes it with **Close**  on the title bar.
- The *Dialog* executes the **Callback Argument** of a **Drop-down List Box** when the user closes it.
- The *Dialog* executes the **Callback Argument** of a **List Box** when the user selects and double-clicks an item in it.
- The *Dialog* executes the **Callback Argument** of a **Text Box** when the user changes its contents and selects another dialog item afterward, clicks in another text box, or clicks **OK**, **Apply** or **Cancel**.
- The *Dialog* executes the **Callback Argument** of a **Button** when the user clicks the **button**.
- The *Dialog* executes the **Callback Argument** of a **Check Box** when the user selects or clears it.
- The *Dialog* executes the **Callback Argument** of a **Radio Button** when the user selects it.
- The *Dialog* executes the **Callback Argument** of a **List View** when the user selects a row in it and double-clicks it.
- The *Dialog* executes the **Callback Argument** of a **Tab Control** when the user selects a tab.
- The *Dialog* executes the **Callback Argument** of a **Menu/Menu Command** when the user selects a menu or a menu command.

Assignment Value

You can assign a value of data type *object*.

Examples

```

MyDialog.CallbackMethod := .Models.Model.&DialogMethod
// this method executes these actions
// .Models.Model.DialogMethod
param CallbackArgument : string
switch CallbackArgument
case "Open"
    print CallbackArgument
case "Apply"
    print CallbackArgument
case "Close"
    print CallbackArgument
case "Text_box"
    print CallbackArgument
case "Button"
    print CallbackArgument
case "Drop_down_list_box"
    print CallbackArgument
case "Group_box"
    print CallbackArgument
case "Check_box"
    print CallbackArgument
case "List_box"
    print CallbackArgument
case "List_view"
    print CallbackArgument
case "Tab_control"
    print CallbackArgument
else
    print "No argument defined: ",CallbackArgument
end

```

```

param action: string
switch action
case "Open" then
    @.setIndex("VariantType", @.VariantNo)
    @.setCheckbox("SunRoof", true)
    @.setValue("Vanity text", "Enter your text")
case "Apply"
    @.VariantNo := @.getIndex("VariantType")
case "Close"
    // no action is required
case "CallbackChart" then mychart.active := true
case "CallbackReport" then myreport.show
end

```

```

param action : string

print "Current action: ",action
switch action
case "Open"

```

```

case "Apply"
case "Close"

case "Item_1_1"
    promptmessage("Action 1")
case "Item_1_2"
    promptmessage("Action 2")
case "Item_2_1"
    promptmessage("Mission 1")
case "Item_2_2"
    promptmessage("Mission 2")
case "X_but"
    promptmessage("Button action")
end

```

```

param action : string

print "Current action: ",action
switch action
case "Open"
    switch Color
    case "Red"
        @.setCheckBox("red_RB",true)
    case "Blue"
        @.setCheckBox("blue_RB",true)
    case "Green"
        @.setCheckBox("Green_RB",true)
    end
    switch Lang
    case "English"
        @.setCheckBox("eng_RB",true)
    case "German"
        @.setCheckBox("ger_RB",true)
    end

case "Apply"
case "Close"

case "red_RB"
    Color := "Red"
case "blue_RB"
    Color := "Blue"
case "Green_RB"
    Color := "Green"

case "eng_RB"
    Lang := "English"
case "ger_RB"
    Lang := "German"

```

```
end
```

```
param action : string

print "Current action: ",action
switch action
case "Open"
    @.setCheckBox("Option_CB",Option)
    @.setList("ListBox",CardFile)
case "Apply"
case "Close"

case "Option_CB"
    if @.getCheckBox("Option_CB")
        Option := true
    else
        Option := false
    end
case "ListBox"
    var n:integer := @.getIndex("ListBox")
    Selection := CardFile[n]
end
```

See also

[Callback Method \[text box\] - Dialog](#)

[Callback Argument \[text box\] - Dialog](#)

DialogX [SimTalk]

Sets the position on the x-axis where *Plant Simulation* shows the dialog box of the *Dialog* designated by <Path>.

Remarks

The unit is a pixel.

The zero point is the top left corner of the screen or the dialog box.

Type

Attribute

Syntax

```
<Path>.DialogX:integer
```

Assignment Value

You can assign a value of data type *integer*.

Specify -1 for either *DialogX* or *DialogY* to center the *Dialog* on screen.

Example

```
MyDialog.DialogX := 350
```

SimTalk

DialogY [SimTalk]

See also

X-Position [dialog item]

DialogY [SimTalk]

Sets the position on the y-axis where *Plant Simulation* shows the dialog box of the *Dialog* you designed when you open it.

Remarks

The unit is one pixel.

The zero point is the top left corner of the screen or the dialog box.

Type

Attribute

Syntax

```
<Path>.DialogY:integer
```

Assignment Value

You can assign a value of data type *integer*.

Specify -1 for either *DialogX* or *DialogY* to center the *Dialog* on screen.

Example

```
MyDialog.DialogY := 300
```

SimTalk

[DialogX \[SimTalk\]](#)

See also

[Y-Position \[dialog item\]](#)

Locked [SimTalk]

Sets what *Plant Simulation* opens when the user double-clicks the icon of the *Dialog* designated by <Path>:

Type

Attribute

Syntax

```
<Path>.Locked: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

- Specify `true` to lock the dialog box of the object *Dialog*, preventing the user from making changes to the layout of the dialog. A double-click opens the *Dialog*.
- Specify `false` to unlock the *Dialog* and open the dialog box when the user double-clicks the icon, so he can modify the layout of the dialog.

Example

```
MyDialog.Locked := true  
MyDialog.openDialog
```

OpenModal [SimTalk]

Sets if the dialog box of the *Dialog* designated by <Path> will be opened modal (`true`) or not modal (`false`).

Remarks

Modal means that the user cannot open any other *Plant Simulation* dialog windows until he closes the window of the user-defined dialog.

Not modal means that the user can open other *Plant Simulation* boxes when the dialog box of the dialog is open.

Type

Attribute

Syntax

```
<Path>.OpenModal : boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
.Models.MyPlant.MyDialog.OpenModal := true
```

See also

[Open Modal \[check box\]](#)

ShowStandardButtons [SimTalk]

Sets if the *Dialog* designated by <Path> shows the standard buttons **OK**, **Cancel** and **Apply** (`true`) or not (`false`).

Type

Attribute

Syntax

```
<Path>.ShowStandardButtons:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
Dialog.ShowStandardButtons := false
```

See also

[Show Default Buttons \[check box\]](#)

Checkbox

Checkbox [object]

Use the object *Checkbox* for toggling between the states on and off, for toggling operating modes, etc.

Image



Description

The *Checkbox* switches between the states *on* and *off*, it switches operating modes, it switches between runtime mode and debug mode, etc.

By default, the *Checkbox* looks like this: